Field Hours

# CSCE 611 Operating Systems

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 40 minutes

## Processes and the kernel boundary

A process is a program in execution: an address space, open files, and a saved set of registers. The kernel is the program that owns the machine and lends it out. A system call is the legal crossing. The user program puts a number in a register and traps. The hardware switches to kernel mode and a kernel stack. The kernel checks the arguments, does the work, and returns. A jump into the kernel's memory is not a system call. It is a bug the hardware should refuse.

The address space is the protection boundary you can draw. Each process has its own page table. A load in process A cannot name a frame that belongs to process B unless the kernel mapped it on purpose, for a shared page. That mapping is a decision, not a leak. The history of kernels is a history of where this boundary sat: in a library, in a monolithic kernel, or in a small kernel with servers in user mode.

A monolithic kernel runs file systems, drivers, and scheduling in one address space. A call from one subsystem to another is a function call, which is fast, and a fault in a driver can take the machine down. A microkernel keeps the mechanism, such as address spaces and interprocess messages, in the kernel, and puts services in processes. A call becomes a message. The case study is the trade, not a brand: fewer bugs that kill the machine, more crossings of the boundary.

Resource allocation starts here. The kernel, not the process, decides who runs, which frames exist, and which file blocks are real. A process that appears to own the processor owns a time slice the scheduler can take back. The rest of the course is the policy behind that revocation: scheduling, memory, locks, and the disk.

The trap is the only door. Below it, the kernel can see every process. Above it, a process cannot.

User process

Trap, with a call number

Kernel, checking arguments

The device or the disk

### A read system call

1. The process puts the file handle, the buffer pointer, and the length in the agreed registers and traps.

2. The kernel checks that the handle is open and that the buffer is inside the process's address space.

3. It copies bytes into that buffer, or arranges for them to arrive, and returns a count. The process does not touch the disk controller.

Problem 1. A user program writes directly to the physical address of the disk's command register. What should happen on a machine with user and kernel mode?

Answer. The access should fault. That address is not mapped into the user page table. If it is mapped writable, the kernel's protection is already broken.

Problem 2. Name one reason a microkernel message is slower than a monolithic function call, and one failure it contains.

Answer. The message crosses address spaces: a trap, a copy or a map, and a scheduling decision. A fault in the file-server process does not automatically overwrite the scheduler's memory. The cost is the crossing. The containment is the separate address space.

Lecture 2 · 45 minutes

# Threads and scheduling

A thread is a register state and a stack that shares an address space with other threads. Two threads in one process can corrupt each other's data without a trap, because the hardware does not see the sharing as a crossing. The kernel may or may not know about the threads. A kernel thread is scheduled by the kernel. A user thread that the kernel cannot see blocks the whole process when one of them waits for disk. That distinction is the practical content of 'lightweight'.

First-come first-served runs each job to completion. It is simple and it is cruel to a short job stuck behind a long one. Round robin gives each ready thread a quantum and moves it to the tail. A short quantum feels responsive and spends more of the machine on context switches. A long quantum turns into first-come. The switch saves registers and changes the page table. That cost is real time, not a diagram.

Turnaround is finish time minus arrival. Response is the time until the job first runs. They are different objectives. Round robin often improves response and can worsen turnaround, because every job stretches by waiting its turn. A scheduler that optimizes the number you did not care about is not a bug in the implementation. It is the wrong policy.

Priority and multi-level feedback are the same idea with memory. A job that uses its whole quantum is suspected of being long and is demoted. A job that blocks for input is suspected of being interactive and is kept high. The suspicion can be wrong, and a priority policy without a bound on starvation can leave a low queue waiting forever. Aging, or a periodic boost, is the repair you should be able to name.

Two jobs, four units each, arriving together. The quantum is one.

First come

A finishes at 4

B finishes at 8

Average turnaround 6

Round robin

A finishes at 7

B finishes at 8

Average turnaround 7.5

### Why round robin stretched the average

1. First-come runs A for four units, then B for four. Finish times 4 and 8. Average 6.

2. Round robin alternates A, B, A, B, A, B, A, B. A's fourth unit is the seventh slot. B's fourth unit is the eighth.

3. Finish times 7 and 8. Average 7.5. Both jobs ran early. Both finished later than the better of the two first-come finishes, and the average got worse.

Problem 1. Job A is 100 units, job B is 1 unit, B arrives just after A starts, quantum is 100. What are the finish times if A started at time 0?

Answer. The quantum is long enough that A runs to completion. A finishes at 100. B finishes at 101. This round robin behaved as first-come. The quantum, not the name of the policy, did that.

Problem 2. A user-level thread blocks in a page fault and the kernel only scheduled the process. What happens to the other user threads?

Answer. They stop. The kernel descheduled the only entity it knows about. A kernel-visible thread would let the others run. That is the operational difference.

Lecture 3 · 45 minutes

## Locks, condition variables, and deadlock

Shared mutable state needs a lock if two threads can touch it. A lock's contract is mutual exclusion: one holder. The implementation on a single processor can disable interrupts for a few instructions. On several processors you need an atomic instruction, such as compare-and-swap or a test-and-set, so two cores cannot both observe a free lock and both take it. A lock that is wrong under two cores and right under one is not a lock. It is a coincidence.

A condition variable lets a thread wait for a state without spinning. The pattern is: hold the lock, while the condition is false wait on the variable (which releases the lock and reacquires it), then act, then signal. The while, not an if, is required. A signal is a hint. The condition may be false again by the time you run, because another thread woke first. An if-wait loses that race.

Deadlock is a cycle of threads each holding a resource the next one wants. Four conditions are usually named: mutual exclusion, hold and wait, no preemption, and circular wait. Break any one and this particular deadlock cannot form. The ordinary engineering break is an order: every thread acquires the locks in the same global order, so a cycle cannot close. The order has to be written down. Two correct locks in opposite orders are a deadlock.

A case you should be able to draw: thread 1 holds lock A and wants B, thread 2 holds B and wants A. Neither can proceed. Detection builds a waits-for graph and looks for a cycle. Prevention refuses the allocation that would create one, or uses the lock order so the cycle is unreachable. Killing a thread is a recovery, and it is a poor API.

The wait pattern. The while is the part people delete, and then the bug is rare.

1. Acquire the lock

2. While the state is wrong, wait

3. Use the state

4. Unlock, and signal if you changed it

### Two locks, two threads

1. Thread 1 locks A, then tries B. Thread 2 locks B, then tries A.

2. Each holds what the other wants. The cycle is A to thread 1 to B to thread 2 to A.

3. If both threads lock A before B, the second thread waits at A and the first can finish and release B. No cycle.

Problem 1. A thread checks a queue, sees it empty, then waits. Another thread enqueues and signals between the check and the wait. What is wrong, and what is the repair?

Answer. The signal is lost if it happens before the wait is registered, and the waiting thread sleeps forever. The check and the decision to wait must hold the same lock the signaler holds. Use the while-wait pattern under that lock.

Problem 2. Three threads lock pairs among locks A, B, and C, each in increasing letter order. Can they deadlock on these locks?

Answer. No. A global order means any waiter holds a higher lock and wants a lower one only if someone broke the order. With everyone taking lower letters first, the waits-for edges point the same way and cannot cycle.

Lecture 4 · 40 minutes

## Page faults and frames

The architecture lecture gave you the trap. This lecture is the policy after the trap. The kernel picks a frame, writes the old page out if it is dirty, reads the new page, updates the page table, and returns to the instruction. The instruction restarts. It does not resume in the middle of a partially completed store. That restart rule is why the hardware and the kernel have to agree on precise faults.

Replacement is which frame to take. Least recently used is a good policy and an awkward one to implement exactly, because touching a page on every load is expensive. A clock approximates it: a hand sweeps frames, a reference bit is set by hardware when the page is used, and the hand clears the bit and skips a page that was used. A page the hand finds with a clear bit is the victim. It is not least recently used. It is cheap and usually close.

Thrashing is the failure mode. The working set of pages that are in active use is larger than the frames you have, so every reference faults, and the disk does nothing but replace pages that will be needed immediately. Adding a process can reduce the useful work, because every process gets fewer frames. The repair is to run fewer processes, or to give the busy one more frames, not to schedule harder.

Allocation across processes is the same shortage. A global replacement pool lets one greedy process steal frames from a careful one. A local pool gives each process a budget and lets it thrash alone. Working-set policies try to measure the set and refuse to run a process whose set does not fit. You should be able to say which pool a fault charges.

A fault on a present instruction. The instruction runs again after the map is fixed.

1. Trap on the missing page

2. Choose a frame

3. Write back if dirty, then read

4. Update the table and restart

### Clock on four frames

1. Reference bits are 1, 1, 0, 1, and the hand is on the first frame.

2. The hand clears the first bit, clears the second, and finds the third already 0. Frame 3 is the victim.

3. A true least-recently-used choice might differ. The clock did one pass and no per-load timestamp.

Problem 1. A page is clean and chosen as victim. Do you write it to disk?

Answer. No. A clean page matches disk already. You may reuse the frame. A dirty page must be written first or the changes disappear.

Problem 2. CPU utilization falls as you add the fifth process, and the disk is busy with page traffic. What is the name, and what do you do?

Answer. Thrashing. Reduce the number of runnable processes or increase frames for the working set. Raising the scheduler's quantum does not create memory.

Lecture 5 · 45 minutes

## Files, crashes, and a case study

A file is a named sequence of bytes with an owner and a length. The inode, or its equivalent, holds the metadata and the map from logical block to physical block. Directory entries are names pointing at inodes. Two names for one inode are a link, and the inode's link count is what keeps the bytes alive. Deleting a name is not deleting the bytes if another name, or an open file, remains.

A crash in the middle of a rename or a file extension can leave the map and the bitmap disagreeing: a block marked free and also reachable, or a block marked used and reachable from nowhere. File-system check walks every inode and rebuilds the bitmap. It can run for a long time, and it can only keep the structures consistent. It cannot know which of two versions of a user-level record you wanted.

A journal is a specialized repair. The system writes a note describing the metadata change, then does the change, then marks the note done. After a crash, the kernel replays finished notes and drops unfinished ones. Ordered and data-journaling modes differ on whether the file's bytes are in the note or only the map is. The promise is about the metadata unless you paid for the bytes.

Put the case studies side by side. A monolithic kernel's file system is a set of functions that share the disk driver and can lock the same buffers. A microkernel's file system is a process that receives read and write messages and answers them. The on-disk inode can be identical. The crash story is about the disk, and it does not care which address space built the journal record. The concurrency story does care: the locks are either inside one kernel or they are messages that cannot race inside one address space in the same way.

Names and bytes are different objects. The link count is what joins them.

Directory entry, a name

Inode, metadata and block map

Data blocks

Bitmap of free blocks

### Create a file, then crash

1. The kernel allocates an inode and a data block, then writes the directory entry.

2. A crash after the allocation and before the directory entry leaks the inode and the block: used, and unnamed.

3. A checker, or a journal replay that never committed the allocate, returns them to the free set. The user's file is gone either way if the name never landed. Consistency is not durability of the name.

Problem 1. An inode has link count 2. You unlink one name. Is the data freed?

Answer. No. The count drops to 1. The bytes stay reachable through the other name. They are freed when the count hits 0 and no process has the file open.

Problem 2. The journal says a metadata update committed, and the crash happened before the data blocks were written, in a mode that journals metadata only. What can a reader observe?

Answer. The file length or the block map can claim bytes that were never written, so the reader can see old disk contents in a file that looks new. Metadata-only journaling did what it promised. It did not promise the data.

Lecture 6 · 180 minutes

## Address spaces

A process is an address space plus a kernel structure that holds its registers, its open files, and its page table. The address space is a lie the kernel maintains: each process sees a private range of virtual addresses. The kernel switches the page table on a context switch. If it forgets, the next process runs on the previous process's memory.

Fork duplicates the address space. Exec replaces it. A shell that forks and execs is the ordinary creation path. Copy-on-write makes the fork cheap until a write. The child and the parent share pages marked read-only, and a write faults, copies, and continues. A program that writes every page after fork pays the copy anyway.

The kernel boundary is a trap. A system call puts arguments in registers or on the user stack, executes a trap instruction, and lands in a handler that checks the arguments before using them. The check is the security of the call. A pointer passed by the user must be verified to point at user memory. The kernel must not trust it.

A context switch saves registers, switches the page table, and restores the next process's registers. It is not free. A design that switches on every short critical section will spend the machine on saves and TLB refills. Threads in one address space switch cheaper because the page table stays.

Fork, then exec.

Fork duplicates the control block -> Pages are shared copy-on-write -> Exec drops those pages -> The

new image is mapped

### A shell command

1. The shell forks. The child still runs shell code, in a shared address space, until it does something that writes or until it execs.

2. The child execs the program. The shell's pages are unmapped in the child. The parent's pages stay.

3. If exec fails, the child is still a shell and should exit. If it returns to the prompt loop, you have two shells reading the terminal.

Problem 1. Why must the kernel check a user pointer before it copies bytes to it?

Answer. The pointer may be null, unmapped, or aimed at kernel memory. Copying to it without a check crashes the kernel or overwrites kernel state. The check is the difference between a system call and a hole.

Problem 2. Two threads in one process switch. Why is that cheaper than two processes switching?

Answer. They share the page table, so the kernel does not switch address spaces and the TLB need not be flushed for that reason. The register save remains. The expensive part of a process switch is the address space, not the registers alone.

Lecture 7 · 180 minutes

## Scheduling in more detail

The earlier lecture compared FIFO and round robin on three jobs. This week adds the goals. Response time matters for interactive jobs. Turnaround matters for batch jobs. Fairness matters when one user can starve another. No single policy wins all three. Say which one you are optimizing.

Multi-level feedback queues approximate shortest-job-first without being told the length. A new job starts in a high-priority queue with a short quantum. If it uses the quantum, it sinks. Interactive jobs that block stay high. CPU-bound jobs sink and get longer quanta. The policy can starve a job that always uses its quantum if you never boost it. Aging, a periodic boost, is the fix.

Priority inversion is a low-priority job holding a lock that a high-priority job needs, while a medium job runs and the low job never gets the CPU to release the lock. Priority inheritance raises the holder's priority to the waiter's for the duration of the hold. It is a patch on locks, not a reason to ignore them.

On a multiprocessor, a scheduler also chooses a core. Migrating a process drops its cache warmth. Leaving it forever on a busy core ignores an idle one. Affinity has a time limit. Measure the migration cost before you pin everything.

Feedback queues.

New jobs enter at the top

Using a quantum moves you down

Blocking early keeps you up

Aging pulls a stuck job back up

### Three jobs, feedback

1. Jobs of 1, 10, and 10 time units arrive together. Quantum at the top queue is 1.

2. The short job runs its 1 and finishes in the top queue. The two long jobs use their quantum and sink.

3. The short job's turnaround is 1. Under FIFO behind a long job it would have been 11 if a long job had been first. Feedback did not need to be told the length.

Problem 1. A low-priority thread holds a lock, a high-priority thread waits for it, and a medium thread spins on the CPU. What is the inversion, and what does inheritance change?

Answer. The high-priority thread is effectively waiting behind the medium thread, because the lock holder cannot run. Inheritance raises the holder to high priority so it runs, releases, and drops back. The medium thread no longer sits between the lock and the release.

Problem 2. Why can a strict priority scheduler starve a low-priority job even if every job eventually blocks?

Answer. If higher-priority work is always ready, the low job never runs. Blocking elsewhere does not help this job. You need aging or a share of the CPU reserved for it. 'It will block' is a hope about other jobs, not a fairness proof.

Lecture 8 · 180 minutes

## More concurrency

A lock gives mutual exclusion. A condition variable lets a thread wait for a predicate without spinning. The pattern is: hold the lock, while the predicate is false wait on the condition, which atomically releases the lock and sleeps, then recheck the predicate. The recheck is required because the wakeup may be stale, and because another thread may have taken the resource.

A semaphore counts. It is a lower-level tool. You can build a lock and a condition from it, and you can also hide the predicate inside the count and then be unable to say what you are waiting for. Prefer a lock and a condition when the predicate is a data structure. Use a semaphore when the count itself is the resource, such as slots in a buffer.

Deadlock needs four conditions: mutual exclusion, hold and wait, no preemption, and a cycle. Break one. The usual engineering break is a lock order: every thread acquires locks in the same global order, so a cycle cannot form. Timeouts are a recovery, not a prevention. They can leave the data structure half-updated if you are not careful about what the timeout aborts.

A race is a missing lock, not a deadlock. The symptom is a rare wrong answer. Adding locks until it goes away can add a deadlock. Name the invariant the lock protects before you add it.

Wait for a predicate.

1. Acquire the lock

2. While the predicate is false, wait

3. The wait releases the lock

4. On wake, recheck under the lock

### A one-slot buffer

1. The predicate for the consumer is 'the slot is full.' The producer waits on 'the slot is empty.'

2. The consumer takes the item, marks the slot empty, and signals the producer. If the signal happens before the producer waits, a condition variable without a predicate check loses the wakeup. The while-loop recheck, together with holding the lock across the test, closes that hole. The lock is why the signal cannot sneak between the test and the wait.

3. Two consumers without the while can both pass a single if, and the second reads an empty slot. The while is the second consumer's protection.

Problem 1. Thread A locks X then Y. Thread B locks Y then X. How do you remove the deadlock without removing either critical section?

Answer. Impose an order. Both threads lock X before Y. A cycle needs opposing orders. Timeouts are the alternative and they complicate the failure path. The order is the prevention.

Problem 2. You replace the while around a condition wait with a single if. Describe a wakeup that breaks the code.

Answer. A thread is signaled, another thread runs first and falsifies the predicate, and the woken thread proceeds without rechecking. Or two threads are woken for one resource. The if assumes the predicate is still true. It is not, once more than one waiter exists or a signal can be stolen.

Lecture 9 · 180 minutes

## Replacement and thrashing

A page fault on an unmapped page is how demand paging fills memory. Replacement chooses the victim. The optimal policy evicts the page used farthest in the future. You cannot implement it without the future, and you can use it as a yardstick on a trace.

LRU evicts the least recently used page. It works when the recent past matches the near future, which is the locality assumption. It fails on a scan larger than memory: the scan evicts everything useful and never reuses the pages it just brought in. Clock approximates LRU with a reference bit and a sweeping hand, and it has the same scan failure.

Thrashing is the state where the system spends its time faulting because the multiprogramming level is too high. Each process's working set is reasonable and the sum is not. The fix is to run fewer processes, or to refuse admission, not to tweak the clock hand. A process whose working set exceeds RAM will thrash alone. No replacement policy saves it.

The working set of a window is the set of pages referenced in that window. If you size the window and admit a process only when its working set fits in the free frames, you are doing the admission control thrashing requires. Counting faults after the fact is how you notice you did not.

Clock on four frames.

Hand points at a frame

Reference bit 1 becomes 0, hand moves

Reference bit 0 is the victim

A scan sets every bit and then evicts everything

### A loop over five pages, four frames

1. Pages 1, 2, 3, 4, 5, then repeat. Four frames.

2. After the first four faults the frames hold 1 through 4. Touching 5 evicts 1 under FIFO or LRU. Touching 1 evicts 2. Every access faults.

3. This is Belady's classic small example of a working set that does not fit. A smarter policy among these four frames still faults every time around the loop. You need a fifth frame or a different access pattern.

Problem 1. Optimal replacement on a trace faults 10 times and LRU faults 12. What do you conclude about LRU on this trace?

Answer. It is close to the yardstick. Two extra faults are not a reason to invent a new policy. If LRU had faulted 100 times, the trace lacks the locality LRU assumes, and you look at the access pattern.

Problem 2. Each of eight processes has a working set of half of RAM. They are all runnable. What happens, and what is the control?

Answer. The sum of working sets is four times RAM, so the machine thrashes. Admission control should keep the sum of working sets under RAM, which means running one or two of these processes, not eight. Replacement policy is the wrong knob.

Lecture 10 · 180 minutes

## File systems

A file is a name, metadata, and data blocks. A directory maps names to inode numbers. An inode holds the size, the permissions, and the block pointers. Separating the name from the inode is why a file can have two names, and why deleting a name does not free the blocks until the last name and the last open handle are gone.

A crash can leave a metadata update half done: the inode points at a block that was never marked allocated, or a directory entry points at an inode that was freed. Fsck walks the structures and repairs what it can, slowly. A journal writes the intended metadata update to a log, and recovery replays or discards the log. The data blocks may still be stale unless the journal includes them or the file system orders the writes.

The inode and the bitmap must agree. The usual order writes the bitmap first or logs both. A crash between the two writes is the case you must be able to describe. 'The file system is journaled' is not a description of that order.

Performance comes from locality of the data blocks and from a cache of inodes and directories. A workload that creates a million tiny files stresses the metadata path, not the sequential bandwidth. Measure the workload you have.

Create a file, then crash.

1. Allocate an inode

2. Allocate a block

3. Point the inode at the block

4. Add the directory entry

5. A crash can sit between any two

### Crash after the directory entry

1. The directory entry is on disk, the inode is on disk, and the data block was never written. The file exists and reads as zeros or as old block contents.

2. A journal of the metadata can make the create atomic with respect to the directory. It does not by itself contain the user's bytes.

3. If the application needs the bytes durable, it fsyncs the file and then the directory, in that order, or it uses a file system mode that journals data. The call is part of the contract.

Problem 1. Two names point at one inode. You unlink one name. When are the blocks freed?

Answer. When the link count hits zero and no process still has the file open. The other name still owns the inode. Unlink removes a name, not necessarily the bytes.

Problem 2. The bitmap says a block is free and an inode still points at it. Which repair is safe?

Answer. Trust a defined order, or mark the block allocated if the inode is live. Marking it free while the inode points at it will hand the same block to a new file. Fsck's job is to pick the rule and apply it. A journal avoids reaching this state.

Lecture 11 · 180 minutes

## Disks and the page cache

A rotating disk pays a seek and a rotational delay, milliseconds, before a transfer that is fast. Random 4-kilobyte reads are seek-bound. Sequential reads are transfer-bound. A flash device does not seek, and it still punishes small random writes because of erase blocks. The lesson is not 'disks are slow.' It is that the access pattern dominates the device.

The page cache holds file pages in memory and writes them back later. A write that returned from write() is not durable. It is durable after a flush that the device has acknowledged. Applications that need a crash to preserve the write must flush, and they must know what the flush covers.

Read-ahead helps sequential scans and hurts random reads if it evicts a useful page for a page nobody will touch. The cache does not know the future. A hint, or a detected sequential run, is how it guesses.

RAID and a single device differ in the failure story. Striping without redundancy increases bandwidth and the chance that one disk kills the volume. Mirroring trades capacity for survival of one disk. The file system journal does not replace a mirror. They handle different failures.

Two access patterns.

Sequential

One seek, then a long transfer

Read-ahead pays

Bandwidth limited

Random 4 KB

A seek per read

Read-ahead wastes the cache

IOPS limited

### One hundred random reads

1. A disk with a 5-millisecond average seek does 100 random reads in about half a second, plus transfer, which is noise next to the seeks.

2. The same bytes sequential are one seek plus a short transfer, tens of milliseconds.

3. An algorithm that 'just reads the file' can be either, depending on whether the layout is sequential. The file system allocator is why create-order often matches read-order, and why it stops matching after fragmentation.

Problem 1. write() returned and the power failed before a flush. What can the application assume is on disk?

Answer. Nothing about that write. The page cache may have held it. Durability starts at the flush acknowledgement, and only for what that flush covered.

Problem 2. Why can a flash device be fast at random reads and still slow at small random writes?

Answer. Reads do not erase. A small write may force an erase of a much larger block and a copy of the surviving pages. The read path and the write path are different machines inside the same package.

Lecture 12 · 180 minutes

## The kernel as a protector

The kernel mediates every resource. A bug in that mediation is a privilege bug. Time-of-check to time-of-use is the pattern: the kernel checks a file's permissions, the user replaces the file with a link, and the kernel then opens the wrong object. The fix is to bind the check to the object, not to the name, and to refuse to follow a swap in between.

Capabilities and access-control lists are two ways to store the permission. An ACL asks, for this object, who may act. A capability is an unforgeable reference that confers the right. The kernel must be able to forge-proof the reference, which is why a capability is not a filename the user typed.

A buffer overrun in the kernel is a full compromise, because the kernel's address space is the machine. User

memory corruption is a compromise of that user. The same bug class, and a different boundary. The checks on system-call arguments are the ordinary way the boundary holds.

Least privilege for a process means it holds only the handles it needs. A daemon that opens a privileged port and then drops its credentials is following that rule. One that keeps the credentials because restarting was annoying is not.

A check that is not a use.

1. Look up the name

2. Check the permissions

3. The user changes the name

4. The use acts on a different object

### A directory and a symlink

1. A privileged program checks that a path is owned by root, then opens it.

2. Between the check and the open, another process replaces a component with a symlink to a file the program should not write.

3. The open follows the symlink and writes the wrong file. Opening with a flag that does not follow the last component, or using a file handle from the check, closes the window. Two separate lookups do not.

Problem 1. Why is a filename an insufficient capability?

Answer. Any process that can guess or read the name can present it, and the name can be rebound to a different inode. A capability must be unforgeable and bound to the object. The kernel issues it. The user does not type it.

Problem 2. A system call copies a path from user memory without checking the pointer. What can the caller make the kernel do?

Answer. Read or write kernel memory, fault on an unmapped address and take down the machine, or supply a path the caller should not have been able to construct. Validate the range before the copy.

Lecture 13 · 180 minutes

## A small kernel, traced

Take one system call and walk it. Read is enough. The user puts a file descriptor, a buffer, and a length in the agreed registers and traps. The handler checks the descriptor, checks the buffer, and calls the file's read operation. The file system copies data into the page cache if needed and then into the user buffer. The handler returns the count or an error. Every step can fail, and the failures are part of the interface.

A context switch in the middle is normal if the read must wait for the disk. The process sleeps, another runs, and the interrupt wakes the sleeper. The invariant is that the kernel lock or the sleep is arranged so the wakeup is not lost. You already saw that pattern on condition variables. The disk interrupt is the signal.

Draw the user memory, the kernel stack, the inode, and the page cache as four boxes and show which code may write which box. If user code can write the inode, the picture is wrong.

This walk is the review for the course. Scheduling, memory, files, and concurrency are all in one read that blocks. If you can narrate it, you can narrate the others.

A blocking read.

Trap -> Check the descriptor and the buffer -> Miss the cache and sleep -> Interrupt wakes the process -> Copy out and return

### Read of 100 bytes, cold cache

1. The page is not cached. The process marks itself sleeping and the scheduler runs someone else. The read has not returned.

2. The disk interrupt arrives. The driver copies the sector into the page cache and wakes the sleeper.

3. The sleeper runs, copies 100 bytes to the user buffer, and returns 100. If the user buffer was invalid, the copy fails and the system call returns an error without having changed the file.

Problem 1. The disk interrupt wakes the process before the process has finished going to sleep. What must the sleep implementation do?

Answer. The same thing a condition variable does. The wakeup must not be lost. The process checks a flag under the same exclusion that the interrupt sets, or it uses a sleep primitive with that guarantee. A bare flag tested before the sleep, with the wake in between, loses the event and sleeps forever.

Problem 2. Name two checks that happen before any byte is copied to the user.

Answer. The descriptor refers to an open file that allows read, and the user buffer is a writable range in the user's address space of the requested length. Either failure returns an error and copies nothing.

Lecture 14 · 180 minutes

## What you can now explain

You should be able to explain a process switch, a lock and a condition, a page fault, a journaled create, and a system-call check. Those are the course. The hours are the explanations written out on a blank page, plus the problems, not a second reading.

Pick one bug from each week and say which invariant it breaks. A lost wakeup breaks the condition. A missing user-pointer check breaks the kernel boundary. A working set larger than memory breaks admission. Naming the invariant is the skill.

The capstone later will ask you to show a crash and a recovery. The file-system week and the read walk are the material. A crash that you cannot place between two specific writes is a story, not a test.

This is original teaching of the standard mechanisms. It is not a commentary on any one teaching operating system. If you implement a small kernel, the invariants above are the ones to test.

Five invariants.

The page table switches with the process

Wait rechecks its predicate

The working set must fit

Metadata updates survive a crash

User pointers are checked

## One bug, one invariant

1. A test fails rarely: two threads took the same ticket from a counter.

2. The increment was not atomic and there was no lock. The invariant was mutual exclusion on the counter. This is a race, not a deadlock and not a scheduling bug.

3. The fix is a lock around the increment, or an atomic increment. Adding a sleep will make the bug less frequent and not gone.

Problem 1. A process uses far more CPU after you add print statements in the scheduler. What did you probably change?

Answer. The cost and perhaps the frequency of context switches, or you serialized on the console lock. The program's algorithm did not change. Measure switch counts before you rewrite the scheduler.

Problem 2. Which week answers a lost wakeup, and what is the one-line fix in the waiting code?

Answer. The concurrency week. Wait in a loop that rechecks the predicate while holding the lock, so a signal cannot be lost between the check and the sleep, and a stolen wakeup cannot be acted on.